Group-15

Longest Increasing Subsequence

TEAM MEMBERS:

D.KOUSHIK:CB.SC.U4AIE24167

K.SIVA ADITYAN:CB.SC.U4AIE24126

N.SRINIVAS:CB.SC.U4AIE24140

M.JAYESH:CB.SC.U4AIE24128

M.MUKESH:CB.SC.U4AIE24138

Problem Statement For Longest Increasing Subsequence

- Given a sequence of numbers, the goal is to find the longest increasing subsequence using Dynamic Programming.
- The selected elements must maintain the original order of the sequence.
- The elements need not be consecutive, but they must be strictly increasing.
- Since checking all possible subsequences is inefficient, a DP approach is used to store and reuse previously computed results.
- This approach helps in efficiently finding the maximum length of the longest increasing subsequence.

Brute Force Solution for Longest Increasing Subsequence

Idea:

- Generate all possible subsequences of the given array.
- Check which subsequences are strictly increasing.
- Find the maximum length among them

Steps:

- Generate every possible subsequence.
- For each subsequence:
  - Check if it is in increasing order.
  - If yes, note its length.
- Return the largest length found.

Continuation of Brute force for Longest Increasing Subsequence

Why this is Brute Force

- A sequence of length n has 2^n subsequences.
- Each subsequence may need to be checked for increasing order.

Time Complexity

- Generating subsequences: **O(2^n)**
- Checking increasing order: **O(n)**
- Total Time Complexity: **O(n · 2^n)**

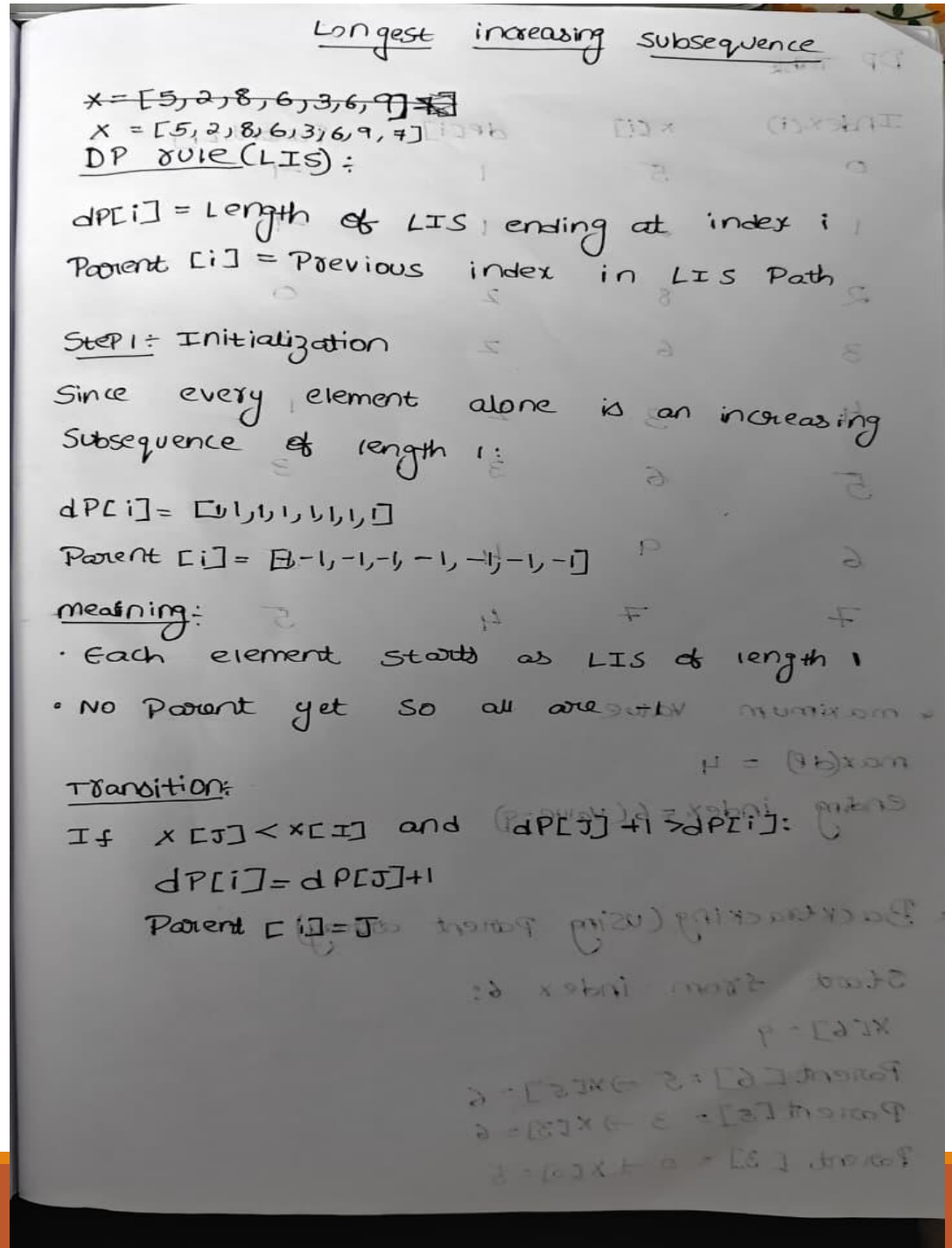
Space Complexity

- Space to store subsequences: **O(n)**
- Total Space Complexity: **O(n)**

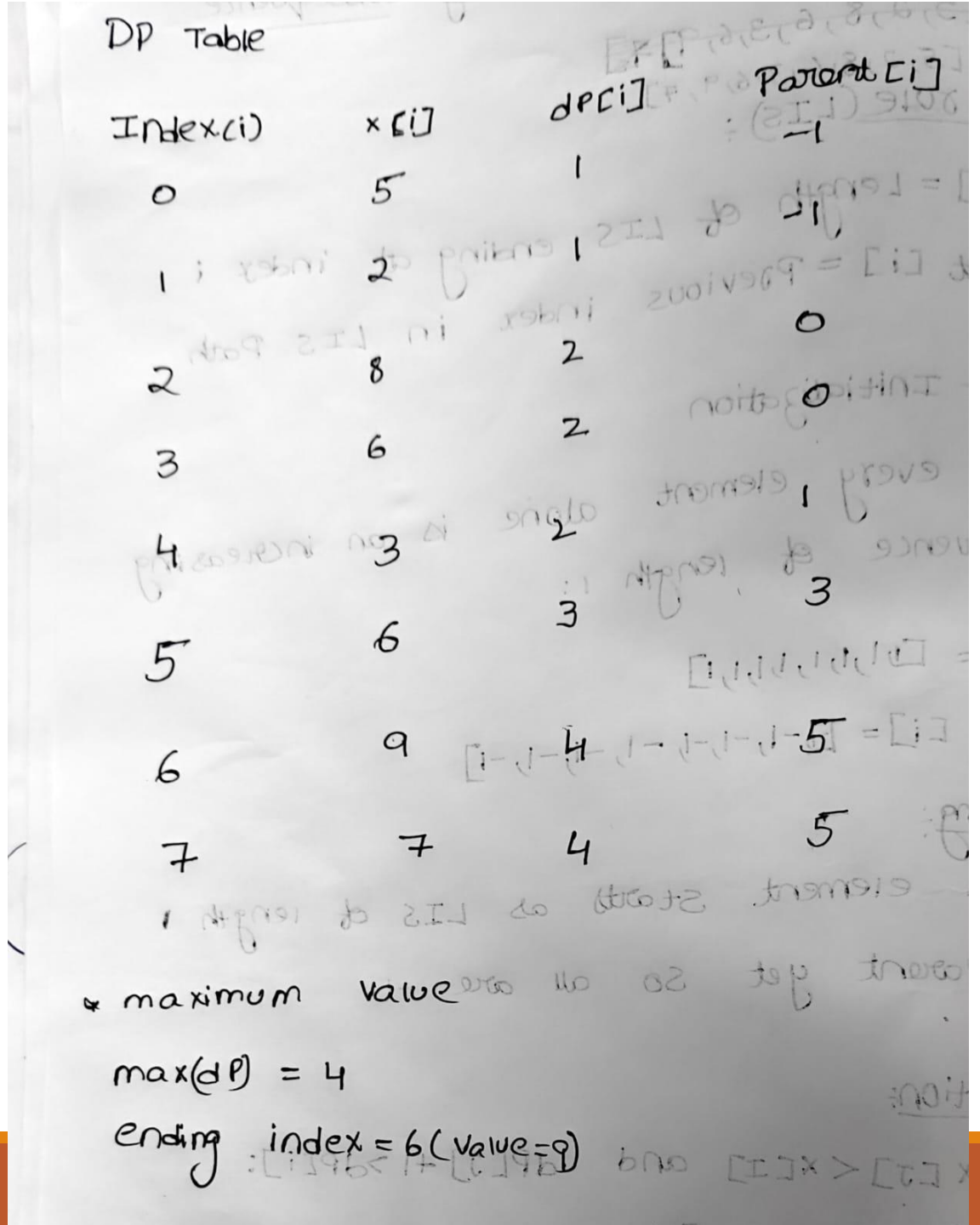
EXAMPLE FOR LIS WITH DP

X=[5,2,8,6,3,9,7]

DP RULE:
dp[i]=Length of LIS ending at index I
Parent[i]=previous index in LIS path



EXAMPLE FOR LIS WITH DP



DP Table

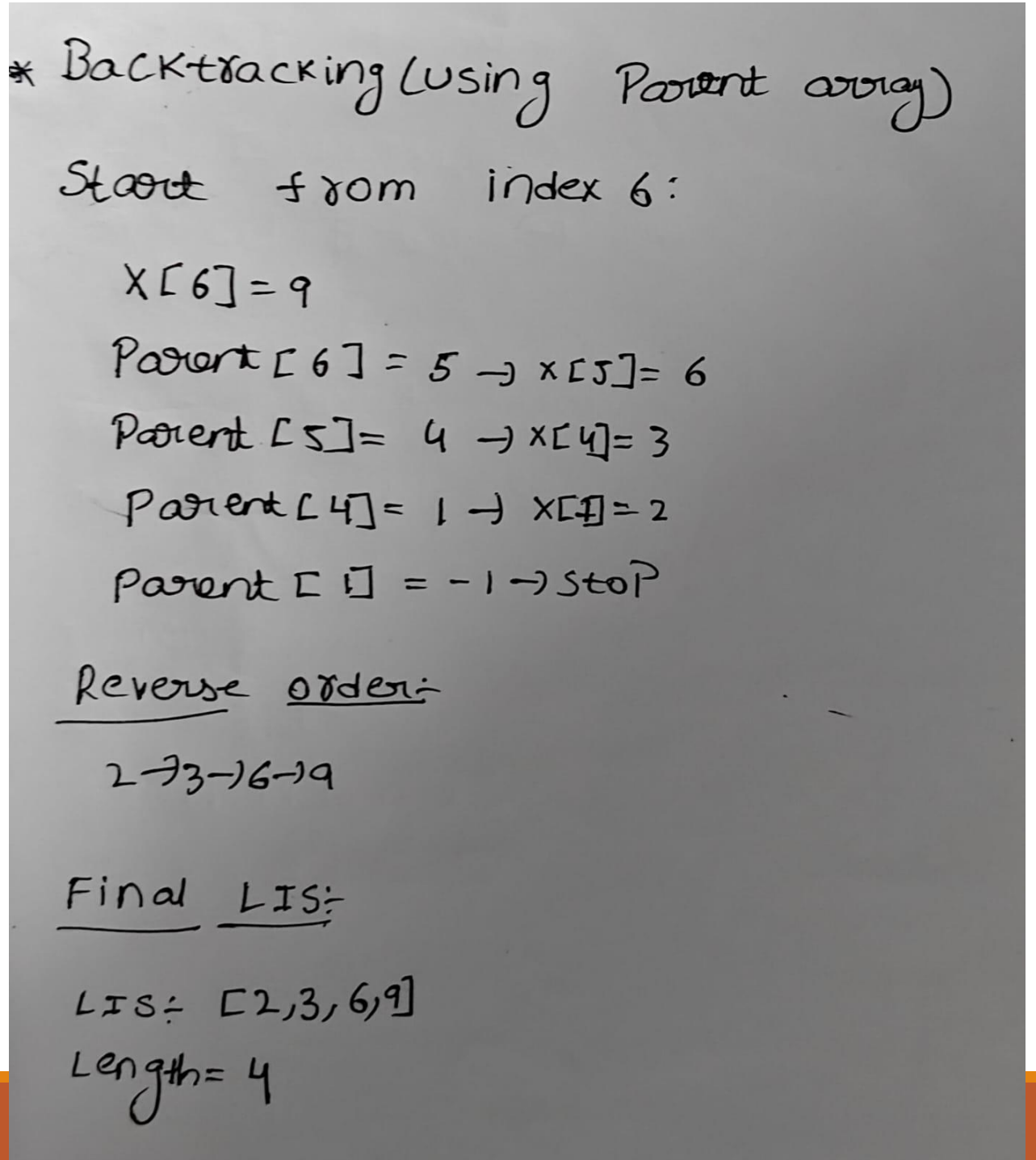
Index(i)	x[i]	dp[i]	Parent[i]
0	5	1	-1
1	2	1	-1
2	8	2	0
3	6	2	0
4	3	2	1
5	6	3	3
6	9	4	5
7	7	4	5

* maximum value

max(dp) = 4

ending index = 6 (value = 9)

EXAMPLE FOR LIS WITH DP



EXAMPLE FOR LONGEST INCREASING SUB SEQUENCE

Input Array :
A = [5, 2, 8, 6, 3, 6, 9, 7]
One Longest Increasing Subsequence:
[2, 3, 6, 9]
Length of LIS: 4
Multiple increasing subsequences may exist. Only maximum length is considered.

Longest increasing subsequence using memoization

Problem: Longest Increasing Subsequence (LIS)

Given array:

```
A = [10, 9, 2, 5, 3, 7, 101]
n = 7
```

Find longest **increasing subsequence**.

(Subsequence → not necessarily continuous)

Memoization Idea

We define:

`dp[i] = length of LIS ending at index i`

To compute `dp[i]`,

we check all previous indices `j < i`.

EXAMPLE CONTINUATION

If:
A[j] < A[i]
Then:
dp[i] = max(dp[i], 1 + dp[j])
If no smaller element exists:
dp[i] = 1

Memoization Structure

- Use recursion
- Store results in `dp[]`
- If already computed → reuse it

Each index is solved once.

Example continuation

dp[0] (10)
No previous element
So:
`dp[0] = 1`

1) dp[1] (9)
Check previous:
10 > 9 →wrong
So:
`dp[1] = 1`

2)dp[2] (2)
10 > 2 →wrong
9 > 2 →wrong
`dp[2] = 1`

3) dp[3] (5)
Check previous:
10 > 5 →wrong
9 > 5 →wrong
2 < 5 →correct
So:
`dp[3] = 1 + dp[2] = 2`

EXAMPLE CONTINUATION

5)dp[4] (3)
Check previous:
10 > 3 →wrong
9 > 3 →wrong
2 < 3 →correct
So:
`dp[4] = 1 + dp[2] = 2`

6)dp[5] (7)
Check previous:
2 < 7 →correct → 1+1=2
5 < 7 →correct → 1+2=3
3 < 7 →correct → 1+2=3
Maximum:
`dp[5] = 3`

dp[6] (101)
Check previous:
10 < 101 → 1+1=2
9 < 101 → 2
2 < 101 → 2
5 < 101 → 3
3 < 101 → 3
7 < 101 → 4
Maximum:
`dp[6] = 4`

Longest increasing subsequence using Tabulation

Example:
A = [10, 9, 2, 5, 3, 7, 101]
n = 7

Tabulation Idea
We define:
dp[i] = length of LIS ending at index i
Initialization:
All dp[i] = 1
Because every element alone is an LIS of length 1.

Filling Rule (Bottom-Up):

For each index i from left to right:

Check all previous indices j < i

If:

A[j] < A[i]

Then:

dp[i] = max(dp[i], 1 + dp[j])

Step-by-Step Table Filling:

Array:

Index: 0 1 2 3 4 5 6

Value:10 9 2 5 3 7 101

Initial:

dp = [1, 1, 1, 1, 1, 1, 1]

Backtracking in Tabulation:

Start from index with maximum dp value:

dp[6] = 4

Find previous smaller element

with dp = 3 → 7

Then dp = 2 → 5

Then dp = 1 → 2

Final LIS:

2, 5, 7, 101

Time Complexity

Outer loop → n

Inner loop → up to n

Total:

$O(n^2)$

Space Complexity

DP array size:

$O(n)$

No extra 2D table needed.

Complexties

Time Complexity:
Outer loop → n
Inner loop → up to n
Total:
$O(n^2)$

Space Complexity:
DP array size:
O(n)
No extra 2D table needed.

EXAMPLE CONTINUATION

FINAL DP TABLE

Index	Value	dp[i]
0	10	1
1	9	1
2	2	1
3	5	2
4	3	2
5	7	3
6	101	4

Final Answer

Maximum value in dp[] =4

Longest Increasing Subsequence length = **4**

Backtracking:

Start from index with maximum dp value:

`dp[6] = 4`

Go backward:

Find element smaller than 101

with dp = 3 → 7

Then find dp = 2 → 5

Then dp = 1 → 2

So sequence:

`2, 5, 7, 101`

Complexties

Time Complexity:

For each index we check all previous indices.

Total comparisons:

$1 + 2 + 3 + ... + n$

$= O(n^2)$

So,

Time Complexity = $O(n^2)$

Space Complexity:

DP array size = n

Recursion stack = n

So,

Space Complexity = O(n)

Table comparison between the two solutions

Approach	Time Complexity	Space Complexity	Explanation
Brute Force	**O(n · 2^n)**	**O(n)**	Generates all subsequences (2^n) and checks each for increasing order
Dynamic Programming	**O(n^2)**	**O(n)**	Stores LIS length ending at each index and reuses results

Observation

- Brute force becomes impossible for large `n`
- DP is much faster and practical

CODE FOR LONGEST INCREASING SUBSEQUENCE USING BRUTE FORCE

```python
import pandas as pd
import itertools
import time

def is_increasing(seq):
    return all(seq[i] < seq[i+1] for i in range(len(seq)-1))

def lis_bruteforce(arr):
    n = len(arr)
    max_len = 0
    best_seq = []

    for r in range(1, n+1):
        for subseq in itertools.combinations(arr, r):
            if is_increasing(subseq):
                if len(subseq) > max_len:
                    max_len = len(subseq)
                    best_seq = list(subseq)

    return max_len, best_seq

file_name = "input_10.xlsx"   # SMALL only
data = pd.read_excel(file_name)
arr = data.iloc[:, 0].tolist()

start = time.time()
length, subsequence = lis_bruteforce(arr)
end = time.time()

print("LIS Length (Brute Force):", length)
print("LIS Subsequence:", subsequence)
print("Time taken:", end - start, "seconds")
```

PYTHON CODE FOR LONGEST INCREASING SUB SEQUENCE USING DP

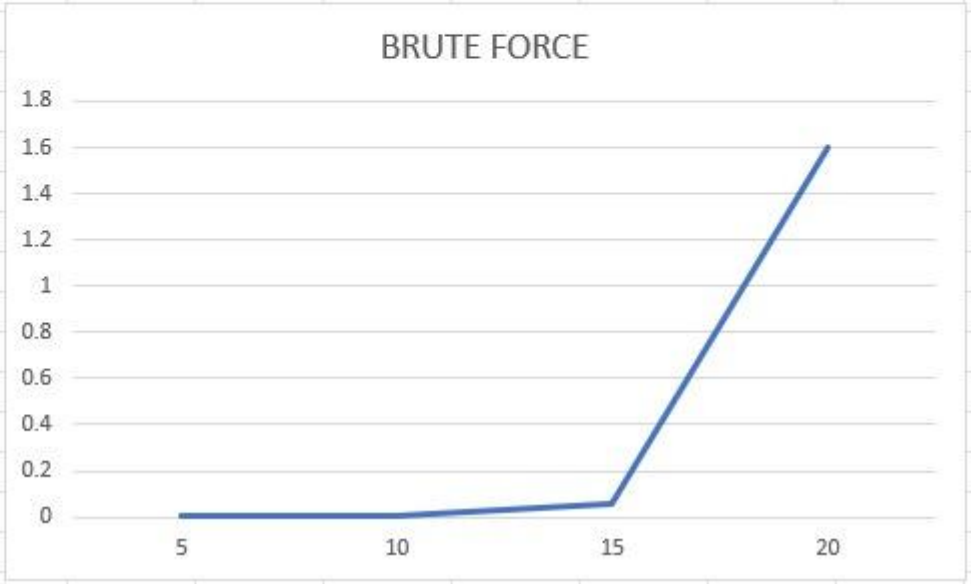
```python
import pandas as pd
import time

def lis_dp(arr):
    n = len(arr)
    L = [1] * n
    prev = [-1] * n

    for j in range(n):
        for i in range(j):
            if arr[i] < arr[j] and L[i] + 1 > L[j]:
                L[j] = L[i] + 1
                prev[j] = i

    max_len = max(L)
    idx = L.index(max_len)

    lis = []
    while idx != -1:
        lis.append(arr[idx])
        idx = prev[idx]
    lis.reverse()
    return max_len, lis

file_name = "input_1000.xlsx"
data = pd.read_excel(file_name)
arr = data.iloc[:, 0].tolist()

start = time.time()
length, subsequence = lis_dp(arr)
end = time.time()

print("LIS Length (DP):", length)
print("LIS Subsequence:", subsequence)
print("Time taken:", end - start, "seconds")
```

PYTHON CODE FOR LONGEST INCREASING SUB SEQUENCE USING Memoization

```python
import pandas as pd
import time
import sys
sys.setrecursionlimit(10**7)
def lis_memo(arr):
    n = len(arr)

    # memo[i] stores LIS ending at index i
    memo = [-1] * n

    def lis_ending_at(i):
        # Base case
        if i == 0:
            return 1

        # If already computed
        if memo[i] != -1:
            return memo[i]

        best = 1  # minimum LIS length

        for j in range(i):
            if arr[j] < arr[i]:
                best = max(best, lis_ending_at(j) + 1)

        memo[i] = best
        return best
    ans = 1
    for i in range(n):
        ans = max(ans, lis_ending_at(i))

    return ans

if __name__ == "__main__":

    # Excel file path (CHANGE ONLY THIS)
    file_path = r"D:\daa\input_30.xlsx"

    # Read Excel file
    df = pd.read_excel(file_path)

    # Convert first column to list
    arr = df.iloc[:, 0].dropna().tolist()

    # Measure time
    start = time.time()
    result = lis_memo(arr)
    end = time.time()
    print("Input Array:", arr)
    print("LIS Length (Memoization):", result)
    print("Time Taken:", end - start, "seconds")
```

BRUTE FORCE TIME COMPLEXITY FOR LONGEST INCREASING SUB SEQUENCE

input	time
5	0.001
10	0.003
15	0.06
20	1.6



DP TIME COMPLEXITY FOR LONGEST INCREASING SUB SEQUENCE

input	time
1000	0.09
20000	26.4
50000	172.5
100000	707.7